

目录

优化版本图传开发问题记录	2
目录	2
1. NVIDIA 驱动版本不匹配	3
问题描述	3
症状	3
诊断命令	3
原因分析	3
解决方案	3
2. PRIME 模式配置问题	3
问题描述	3
症状	3
诊断命令	3
原因分析	4
解决方案	4
3. CUDA 内核模块与用户空间库版本不匹配	4
问题描述	4
症状	4
诊断命令	4
原因分析	4
解决方案	4
4. Unity 编辑器 PlayMode 闪退 (Double Fault)	5
问题描述	5
症状	5
诊断方法	5
原因分析	5
解决方案	5
5. ffmpeg CUDA 初始化失败	6
问题描述	6
症状	6
原因分析	7
解决方案	7
6. Unity 6 Vulkan 模式与 NVIDIA 590 驱动兼容性问题	7
问题描述	7
症状	7
诊断方法	7
崩溃堆栈关键信息	7
原因分析	7
解决方案	8
对 NVDEC 零拷贝方案的影响	8
7. NVDEC 解码帧映射失败 (cuvidMapVideoFrame failed)	8
问题描述	8
症状	9
诊断方法	9
原因分析	9
解决方案	9
关键知识点	10
修改的文件	10
环境信息	10
开发建议	11
相关文件	11
8. 视频帧显示卡住 (decoded 增加但 displayed 停滞)	11
问题描述	11
症状	11
诊断方法	11

原因分析	12
解决方案	12
当前状态	13
进一步诊断 (2026年1月29日更新)	13
修改的文件	15
深入诊断 (2026年1月29日下午更新)	15
修改的文件 (更新)	16
9. cuvidMapVideoFrame 32位/64位函数签名不匹配	16
问题描述	16
症状	17
诊断日志	17
原因分析	17
解决方案	17
加载顺序	18
关键知识点	18
修改的文件	18
10. 函数指针类型与实际加载函数签名不匹配	18
问题描述	18
症状	18
原因分析	19
解决方案	19
完整修改	20
关键知识点	20
修改的文件	20
11. GL对象尺寸与视频实际尺寸不匹配	20
问题描述	20
症状	20
原因分析	21
解决方案	21
关键知识点	22
修改的文件	23

优化版本图传开发问题记录

本文档记录了在开发基于 CUDA-Vulkan 零拷贝的优化图传方案过程中遇到的所有问题、原因分析和解决方案。

目录

1. NVIDIA 驱动版本不匹配
2. PRIME 模式配置问题
3. CUDA 内核模块与用户空间库版本不匹配
4. Unity 编辑器 PlayMode 闪退 (Double Fault)
5. ffmpeg CUDA 初始化失败
6. Unity 6 Vulkan 模式与 NVIDIA 590 驱动兼容性问题
7. NVDEC 解码帧映射失败 (cuvidMapVideoFrame failed)
8. 视频帧显示卡住 (decoded 增加但 displayed 停滞)
9. cuvidMapVideoFrame 32位/64位函数签名不匹配
10. 函数指针类型与实际加载函数签名不匹配
11. GL对象尺寸与视频实际尺寸不匹配

1. NVIDIA 驱动版本不匹配

问题描述

系统中存在多个版本的 NVIDIA 驱动组件，导致 CUDA 功能无法正常工作。

症状

- libcuda.so 链接到 580.126.09 版本
- libnvcuvid.so 链接到 580.126.09 版本
- NVIDIA 驱动实际安装的是 590.48.01 版本
- 运行时出现 CUDA_ERROR_COMPAT_NOT_SUPPORTED_ON_DEVICE 错误

诊断命令

```
# 检查 CUDA 库版本
ls -la /usr/lib/x86_64-linux-gnu/libcuda.so*
ls -la /usr/lib/x86_64-linux-gnu/libnvcuvid.so*
```

```
# 检查已安装的 NVIDIA 包
dpkg -l | grep nvidia.*590
dpkg -l | grep nvidia.*580
```

原因分析

系统进行过 NVIDIA 驱动升级（从 580 到 590），但升级不完整，导致部分用户空间库仍为旧版本。

解决方案

重新安装 590 版本的完整驱动包：

```
sudo apt install -y --reinstall nvidia-driver-590-open \
    libnvidia-compute-590 \
    libnvidia-decode-590 \
    libnvidia-encode-590
```

2. PRIME 模式配置问题

问题描述

双 GPU 笔记本（Intel 集显 + NVIDIA RTX 5080）在使用 PRIME “on-demand” 模式时，Unity 编辑器启动出现严重显示问题。

症状

- Unity 启动时屏幕显示异常（半边壁纸、半边空白）
- ANGLE/EGL 初始化失败日志
- Invalid visual ID requested 错误

诊断命令

```
# 检查当前 PRIME 模式
prime-select query

# 检查默认 GLX 渲染器
glxinfo -B | grep -E "vendor|renderer"
```

原因分析

PRIME “on-demand” 模式下，系统默认使用 Intel 集显进行渲染，而 CUDA/NVDEC 需要 NVIDIA GPU。当 Unity 编辑器（基于 Electron/ANGLE）尝试在 Intel GPU 上初始化图形设备，同时原生插件尝试使用 NVIDIA CUDA 时，产生冲突。

解决方案

切换到 NVIDIA 性能模式：

```
sudo prime-select nvidia  
# 需要注销或重启生效
```

验证切换成功：

```
prime-select query  
# 输出应为: nvidia
```

```
glxinfo -B | grep vendor  
# 输出应包含: NVIDIA Corporation
```

3. CUDA 内核模块与用户空间库版本不匹配

问题描述

安装新版本驱动后未重启系统，导致运行中的内核模块版本与新安装的用户空间库版本不一致。

症状

- ffmpeg 报错: CUDA_ERROR_COMPAT_NOT_SUPPORTED_ON_DEVICE: forward compatibility was attempted on non supported HW
- 原生插件 CUDA 初始化失败
- cuInit(0) 返回兼容性错误

诊断命令

```
# 检查当前加载的内核模块版本  
cat /sys/module/nvidia/version
```

```
# 检查用户空间库版本  
ls -la /usr/lib/x86_64-linux-gnu/libcuda.so.1
```

```
# 检查 DKMS 状态  
dkms status | grep nvidia
```

原因分析

NVIDIA 驱动由两部分组成：1. **内核模块** (nvidia.ko)：系统启动时加载，运行期间不会自动更新 2. **用户空间库** (libcuda.so, libnvcudvid.so 等)：安装后立即生效

当升级驱动后不重启，内核模块仍为旧版本，与新的用户空间库产生 ABI 不兼容。

解决方案

必须重启系统以加载新的内核模块：

```
sudo reboot
```

重启后验证版本一致：

```
cat /sys/module/nvidia/version  
# 应输出: 590.48.01
```

```
ls -la /usr/lib/x86_64-linux-gnu/libcuda.so.1  
# 应链接到: libcuda.so.590.48.01
```

4. Unity 编辑器 PlayMode 闪退 (Double Fault)

问题描述

在 Unity 编辑器中点击播放按钮后，编辑器立即崩溃退出。

症状

- Unity 日志显示: Exiting early due to double fault.
- 项目根目录生成 mono_crash.mem.*.blob 文件
- 崩溃发生在 PlayerLoopController::EnterPlayMode 阶段
- 堆栈中出现 GfxDeviceClient::AcquireThreadOwnership

诊断方法

查找崩溃转储文件

```
find . -name "mono_crash*.blob" -mmin -10
```

查看 Unity Editor 日志

```
find ~/.config/unity3d -name "Editor.log" -mmin -5 | xargs tail -200
```

原因分析

原生 NVDEC 插件在 VideoStreamService.Awake() 中调用 nvp_init()，此时 Unity 正处于从编辑模式切换到 Play-Mode 的过程中：

1. Unity 图形设备状态正在转换
2. GfxDeviceClient 正在获取线程所有权
3. 原生插件尝试初始化 CUDA 上下文
4. CUDA 初始化与 Unity 图形线程产生竞争
5. 触发 double fault（双重故障）导致进程崩溃

解决方案

C# 端修改 (VideoStreamService.cs) 将 NVDEC 初始化从 Awake() 延迟到 Start()：

// 添加标志

```
private bool nvdecDelayedInitDone = false;
```

```
void Start()
```

```
{  
    // 延迟初始化原生 NVDEC: 确保 Unity 图形设备在 PlayMode 下完全就绪  
    if (decodeBackend == DecodeBackend.NativeNvdec && NativeVideoBridge.Available && !nvdecDelayedInitDone)  
    {  
        InitializeNativeNvdec();  
    }  
}
```

```
private void InitializeNativeNvdec()
{
    if (nvdecDelayedInitDone) return;
    nvdecDelayedInitDone = true;

    // ... 初始化代码 ...
}
```

C++ 端修改 (NativeVideoPlugin.cpp) 添加图形设备就绪检查:

```
struct PluginState
{
    std::atomic<bool> initialized{false};
    std::atomic<bool> graphicsReady{false}; // 新增: 追踪图形设备状态
    // ...
};

static void UNITY_INTERFACE_API OnGraphicsDeviceEvent(UnityGfxDeviceEventType eventType)
{
    switch (eventType)
    {
    case kUnityGfxDeviceEventInitialize:
        g_state.graphicsReady.store(true);
        // ...
        break;
    case kUnityGfxDeviceEventShutdown:
        g_state.graphicsReady.store(false);
        // ...
        break;
    }
}

NVP_EXPORT int nvp_init(int width, int height)
{
    // 检查图形设备是否就绪
    if (!g_state.graphicsReady.load()) {
        fprintf(stderr, "[NativeVideoPlugin] nvp_init called but graphics device not ready\n");
        return -10;
    }
    // ...
}
```

5. ffmpeg CUDA 初始化失败

问题描述

当原生 NVDEC 不可用时, 备用的 ffmpeg 管道解码器也无法使用 CUDA 硬件加速。

症状

日志显示:

```
[AVHWDeviceContext] cu->cuInit(0) failed -> CUDA_ERROR_COMPAT_NOT_SUPPORTED_ON_DEVICE
Device creation failed: -542398533.
No device available for decoder: device type cuda needed for codec hevc.
```

原因分析

这是 问题 3 的衍生问题。当 CUDA 驱动版本不匹配时，ffmpeg 也无法初始化 CUDA 设备。

解决方案

解决驱动版本不匹配问题后，ffmpeg 的 CUDA 加速自然恢复正常。

6. Unity 6 Vulkan 模式与 NVIDIA 590 驱动兼容性问题

问题描述

即使在延迟初始化 NVDEC 插件后，Unity 编辑器在 Vulkan 渲染模式下进入 PlayMode 时仍然崩溃。切换到 OpenGL 模式后问题消失。

症状

- Unity 日志显示: Exiting early due to double fault.
- 崩溃堆栈中有 CUDA 相关线程 (cuda00050400142, cuda-EvtHandlr)
- 崩溃发生在 PlayerLoopController::EnterPlayMode → GfxDeviceClient::AcquireThreadOwnership
- 我们的原生插件代码**尚未被调用**（日志中无 NVDEC 初始化信息）
- 切换到 OpenGL 模式后 PlayMode 正常运行

诊断方法

查看 Unity Editor 日志中的线程信息

```
find ~/.config/unity3d -name "Editor.log" -mmin -10 | xargs tail -200
```

强制使用 OpenGL 启动 Unity 进行对比测试

```
/path/to/Unity -force-opengl -projectPath /path/to/project
```

崩溃堆栈关键信息

Thread 5 (Thread ... "cuda00050400142"): # Unity 内部的 CUDA 线程

#0 __GI___poll (...)

#1 ??? () at /lib/x86_64-linux-gnu/libcuda.so.1

Thread 1 (Thread ... "Unity"):

#0 syscall ()

#1 UnityClassic::Baselib_SystemFutex_Wait(...)

#2 Semaphore::WaitForSignal(...)

#3 GfxDeviceClient::AcquireThreadOwnership() # 关键: 获取图形设备线程所有权失败

#4 ScreenManagerLinux::SetWindow(...)

#5 GUIView::UpdateScreenManager()

#6 GUIView::DoPaint(...)

#7 PlayerLoopController::EnterPlayMode() # 进入 PlayMode 时崩溃

原因分析

根本原因: Unity 6 在 Vulkan 模式下内部使用 CUDA 进行某些图形加速操作。当进入 PlayMode 时，Unity 需要重新获取图形设备的线程所有权，但此时 CUDA 线程仍在运行，导致线程同步竞争，触发 double fault。

关键对比:

特性	Vulkan 模式	OpenGL 模式
Unity 内部 CUDA 使用线程模型	☒ 启用 多线程渲染, 复杂同步	☒ 不启用 单线程, 简单
PlayMode 转换	需要复杂的线程所有权转移	简单直接
与 NVIDIA 590 驱动兼容性	☒ 存在问题	☒ 正常

重要说明: 这个崩溃与我们的原生 NVDEC 插件**无关**, 是 Unity 6 + Vulkan + NVIDIA 590 Open Kernel 驱动的兼容性问题。

解决方案

方案 A: 使用 OpenGL 模式 (推荐, 稳定) 强制 Unity 使用 OpenGL 渲染后端:

方法 1: 命令行参数

```
/home/zby/Unity/Hub/Editor/6000.3.5f1/Editor/Unity -force-opengl -projectPath /path/to/project
```

方法 2: 创建启动脚本

```
cat > run_unity_opengl.sh << 'EOF'
```

```
#!/bin/bash
```

```
/home/zby/Unity/Hub/Editor/6000.3.5f1/Editor/Unity -force-opengl -projectPath "$(pwd)" "$@"
```

```
EOF
```

```
chmod +x run_unity_opengl.sh
```

优点: - 稳定, 不会崩溃 - 我们的 NVDEC 插件自动使用 CUDA-GL 互操作路径 - 仍然是 GPU 零拷贝方案

缺点: - 性能略低于 Vulkan (约 5-10%)

方案 B: 等待官方修复

- 升级 Unity 到更新版本 (可能修复此问题)
- 等待 NVIDIA 驱动更新
- 时间不确定

方案 C: 降级 NVIDIA 驱动 尝试降级到 575/570 系列驱动, 可能与 Unity 6 Vulkan 兼容性更好:

```
sudo apt install nvidia-driver-575
```

对 NVDEC 零拷贝方案的影响

渲染模式	互操作方式	零拷贝	性能
Vulkan	CUDA-Vulkan	☒	最优
OpenGL	CUDA-GL	☒	优秀 (略低于 Vulkan)

我们的原生插件同时支持两种路径, 会根据 Unity 当前使用的渲染后端自动选择: - Vulkan 模式 → `nvdec_init_vulkan()` → CUDA-Vulkan 互操作 - OpenGL 模式 → `cuGraphicsGLRegisterBuffer()` → CUDA-GL 互操作

两种方式都实现了 GPU 内的零拷贝传输, 性能差距很小。

7. NVDEC 解码帧映射失败 (cuvidMapVideoFrame failed)

问题描述

在 OpenGL 模式下, NVDEC 初始化成功, CUDA-GL 互操作注册成功, 但视频画面卡住不动。

症状

- 日志显示 [NVDEC] CUDA-GL registration SUCCESS
- 随后反复出现 [NvdecPlugin] cuvidMapVideoFrame failed 或 cuvidMapVideoFrame failed: 205
- 错误码 205 = CUDA_ERROR_INVALID_CONTEXT
- 视频流数据正常接收 ([UdpVideoHandler] 收到切片: frame=xxx)
- 画面完全静止, 无任何更新

诊断方法

查看 Unity Editor 日志中的 NVDEC 相关信息

```
find ~/.config/unity3d -name "Editor.log" -mmin -30 | head -1 | xargs cat | grep -E "cuvidMapVideoFrame|CUDA-GL"
```

原因分析

根本原因: CUVID 解析器的回调函数 HandlePictureDisplay 是在**解析器内部线程**中调用的, 而不是在创建 CUDA 上下文的主线程中。

初始尝试 (不完全正确) 最初尝试使用 cuCtxPushCurrent/cuCtxPopCurrent 来绑定上下文, 但这会返回 CUDA_ERROR_INVALID_CONTEXT (205) 错误。

正确方案 CUVID 提供了专门的上下文锁机制 cuvidCtxLock/cuvidCtxUnlock, 这是为多线程 CUVID 操作设计的, 比手动 push/pop 上下文更可靠。

调用链分析:

主线程:

```
nvdec_init()
→ cuCtxCreate()      # 创建 CUDA 上下文
→ cuvidCtxLockCreate() # 创建 CUVID 上下文锁
→ cuvidCreateDecoder() # 解码器使用上下文锁
→ cuvidCreateVideoParser()
```

解析器内部线程 (由 CUVID 创建):

```
cuvidParseVideoData()
→ HandlePictureDisplay()
→ cuvidCtxLock()      # ☑ 正确: 获取上下文锁
→ cuvidMapVideoFrame() # 现在可以工作
→ cuvidUnmapVideoFrame()
→ cuvidCtxUnlock()    # 释放上下文锁
```

解决方案

在 HandlePictureDisplay 回调函数中, 使用 cuvidCtxLock/cuvidCtxUnlock 而不是 cuCtxPushCurrent/cuCtxPopCurrent:

// 1. 添加函数指针声明

```
tcuvidCtxLock    *cuvidCtxLock = nullptr;
tcuvidCtxUnlock  *cuvidCtxUnlock = nullptr;
```

// 2. 在 InitCvidFunctions() 中加载

```
LOAD_FUNC(cuvidCtxLock)
LOAD_FUNC(cuvidCtxUnlock)
```

// 3. 在 HandlePictureDisplay 中使用

```
static int CUDAAPI HandlePictureDisplay(void* pUserData, CUVIDPARSERDISPINFO* pDispInfo)
{
    auto* ctx = static_cast<NvdecContext*>(pUserData);
```

```

std::lock_guard<std::mutex> lock(ctx->mtx);

if (!ctx->decoder || ctx->width <= 0 || ctx->height <= 0) return 0;

// 使用 cuvidCtxLock 获取上下文访问权限
CUresult lockRes = cuvidCtxLock(ctx->cuLock, 0);
if (lockRes != CUDA_SUCCESS) {
    // 错误处理...
    return 0;
}

// Map decoded surface
CUVIDPROCPARAMS vpp = {};
vpp.progressive_frame = 1;
CUdeviceptr dpSrcFrame = 0;
unsigned int pitch = 0;
CUresult res = cuvidMapVideoFrame(ctx->decoder, pDispInfo->picture_index,
                                   &dpSrcFrame, &pitch, &vpp);

if (res != CUDA_SUCCESS) {
    cuvidCtxUnlock(ctx->cuLock, 0); // 错误时也要解锁
    return 0;
}

// ... 处理解码帧 (NV12→RGB 转换等) ...

cuvidUnmapVideoFrame(ctx->decoder, dpSrcFrame);
cuvidCtxUnlock(ctx->cuLock, 0); // 正常退出时解锁

return 1;
}

```

关键知识点

- cuvidCtxLock vs cuCtxPushCurrent:**
 - cuCtxPushCurrent 是通用的 CUDA 上下文栈操作
 - cuvidCtxLock 是 CUVID 专用的上下文锁，专为解码器多线程操作设计
 - 在 CUVID 回调中应使用 cuvidCtxLock
- 上下文锁的创建:**
 - 在 nvdec_init() 中通过 cuvidCtxLockCreate(&ctx.cuLock, ctx.cuCtx) 创建
 - 解码器创建时需要传入此锁: dci.vidLock = ctx->cuLock;
- 错误码 205 的含义:**
 - CUDA_ERROR_INVALID_CONTEXT 表示当前线程没有有效的 CUDA 上下文
 - 在 CUVID 回调中出现此错误，通常意味着需要使用 cuvidCtxLock

修改的文件

- Assets/Plugins/NativeVideoPlugin/NvdecStub.cpp
 - 添加 cuvidCtxLock 和 cuvidCtxUnlock 函数指针
 - 在 InitCuvidFunctions() 中加载这两个函数
 - 修改 HandlePictureDisplay() 使用 cuvidCtxLock/Unlock

环境信息

项目	值
操作系统	Ubuntu 24.04 LTS
Unity 版本	6000.3.5f1
NVIDIA 驱动	590.48.01 (Open Kernel Module)
GPU	Intel ARL (集显) + NVIDIA RTX 5080 Laptop (独显)
CUDA 版本	13.0 (CUDA Toolkit)

开发建议

1. **驱动升级后必须重启**：任何 NVIDIA 驱动更新后都应重启系统，确保内核模块与用户空间库版本一致。
2. **双 GPU 系统使用 NVIDIA 模式**：对于需要 CUDA/NVDEC 的应用，建议使用 `prime-select nvidia` 切换到性能模式。
3. **延迟初始化原生插件**：在 Unity 中使用原生插件时，应在 `Start()` 而非 `Awake()` 中进行初始化，确保图形设备完全就绪。
4. **添加图形设备状态检查**：原生插件应监听 Unity 图形设备事件，在设备未就绪时拒绝初始化。
5. **保留 fallback 路径**：即使主路径（原生 NVDEC）失败，也应有备用路径（ffmpeg 管道）可用。

相关文件

- `Assets/Plugins/NativeVideoPlugin/NativeVideoPlugin.cpp` - 原生插件主文件
- `Assets/Plugins/NativeVideoPlugin/NvdecStub.cpp` - NVDEC 解码器实现
- `Assets/Scripts/Framework/Video/VideoStreamService.cs` - Unity 视频流服务
- `Assets/Scripts/Plugins/NativeVideoBridge.cs` - P/Invoke 桥接层
- `run_unity_nvidia.sh` - NVIDIA 模式启动脚本

8. 视频帧显示卡住 (decoded 增加但 displayed 停滞)

问题描述

在解决 `cuidMapVideoFrame` 错误后，NVDEC 能够成功解码帧，但视频画面仍然卡住，只显示最初的 1-2 帧。

症状

- 日志显示帧解码正常进行：`decoded=46`，`displayed=2`
- `frames_decoded` 持续增长（视频流接收正常）
- `frames_displayed` 停滞在 2（仅最初 2 帧成功写入 PBO）
- 画面显示第一帧后不再更新
- 无 CUDA 或 OpenGL 错误日志

诊断方法

查看 `RunLog.txt` 中的统计信息

```
tail -f ~/RoboMasterClientWMJ/Log/RunLog.txt | grep -E "decoded|displayed"
```

期望看到类似：

```
# mode=OpenGL, tex=200, pbo=649, cuPbo=174630848, glReady=1, glFailed=0
```

```
# decoded=46, displayed=2, size=1920x1080
```

原因分析

经过分析，问题涉及多个潜在因素：

1. 竞态条件：cuPbo 延迟注册 问题：cuPbo（CUDA-GL 互操作资源）是在 nvdec_get_texture() 中**延迟注册**的（由 Unity 主线程调用），但 HandlePictureDisplay() 回调在**CUVID 内部线程**中执行。

时序问题：

主线程：	CUVID 线程：
nvdec_init()	
↓	
首帧数据到达	HandlePictureDisplay()
↓	→ ctx->cuPbo == nullptr (跳过处理)
nvdec_get_texture()	
→ cuGraphicsGLRegisterBuffer() # cuPbo 注册	
↓	
后续帧	HandlePictureDisplay()
	→ ctx->cuPbo != nullptr (应该成功)

结论：虽然存在竞态，但 displayed=2 说明 cuPbo 注册后有 2 帧成功处理，问题不在于延迟注册本身。

2. cuGraphicsMapResources 静默失败 可能原因：cuGraphicsMapResources() 在成功 2 次后开始返回错误，但原有代码只

现象：- 前 2 次调用成功，frames_displayed++ - 后续调用失败，但没有详细的错误信息

3. frame_ready 标志非原子操作 问题：frame_ready 是普通 bool 而非 std::atomic<bool>，在 CUVID 线程和主线程之间存在数据竞争。

```
// NvdecStub.h
bool frame_ready = false; // 非原子！存在竞态风险

// CUVID 线程写入
ctx->frame_ready = true;

// 主线程读取
if (mut_ctx.frame_ready) { ... }
```

解决方案

步骤 1：添加详细诊断日志 在 HandlePictureDisplay() 中记录 cuGraphicsMapResources 的具体错误码：

```
res = cuGraphicsMapResources(1, &ctx->cuPbo, ctx->stream);
if (res == CUDA_SUCCESS) {
    res = cuGraphicsResourceGetMappedPointer(&dpPbo, &pbo_size, ctx->cuPbo);
}

if (res == CUDA_SUCCESS) {
    // ... 成功处理 ...
} else {
    char msg[128];
    snprintf(msg, sizeof(msg), "cuGraphicsMapResources/GetPointer failed: %d", (int)res);
    throttled_log(msg); // 记录具体错误码
}
```

在 nvdec_get_texture() 中记录纹理上传状态：

```
if (mut_ctx.cuPbo && mut_ctx.frame_ready) {
    static int tex_upload_count = 0;
    // ... glTexSubImage2D ...
}
```

```

    GLenum gl_err = glGetError();
    if (gl_err != GL_NO_ERROR) {
        fprintf(stderr, "[NVDEC] glTexSubImage2D error: 0x%x\n", gl_err);
    }
    tex_upload_count++;
    if (tex_upload_count <= 10 || tex_upload_count % 30 == 0) {
        fprintf(stderr, "[NVDEC] Texture uploaded (count=%d)\n", tex_upload_count);
    }
}

```

步骤 2: 检查 cuPbo 未注册时的行为 在 HandlePictureDisplay() 中添加未注册时的日志:

```

if (ctx->cuPbo) {
    // 正常处理...
} else {
    static int skip_count = 0;
    if (++skip_count <= 5) {
        fprintf(stderr, "[NvdecPlugin] HandlePictureDisplay: cuPbo not registered yet (skip #d)\n", skip_count);
    }
}

```

步骤 3: 将 frame_ready 改为原子变量 (待实施)

```

// NvdecStub.h
std::atomic<bool> frame_ready{false}; // 线程安全

```

当前状态

- ☒ 添加了 cuGraphicsMapResources 错误码日志
- ☒ 添加了 cuPbo 未注册时的诊断日志
- ☒ 添加了纹理上传计数日志
- ☒ 需要重新测试以获取具体错误信息
- ☒ frame_ready 原子化修改待实施

进一步诊断 (2026年1月29日更新)

经过测试, 发现即使使用了 cuvidCtxLock, cuvidMapVideoFrame 仍然返回错误 205 (CUDA_ERROR_INVALID_CONTEXT)。

日志输出:

```

[NVDEC] CUDA-GL registration SUCCESS
[NvdecPlugin] cuvidMapVideoFrame failed: 205
[NvdecPlugin] cuvidMapVideoFrame failed: 205
... (持续失败)
[NVDEC] Texture uploaded (count=1, decoded=26, displayed=2)

```

关键发现: cuvidCtxLock 只获取锁, 不会自动将 CUDA 上下文绑定到当前线程!

问题根源 在 CUDA 13.0 中, cuvidCtxLock 的行为与预期不同: - 它只获取互斥锁, 防止多线程同时访问解码器 - 它**不会**将 CUDA 上下文推送到当前线程的上下文栈

因此, 在 CUVID 回调线程中, 即使获取了锁, 当前线程仍然没有有效的 CUDA 上下文, 导致 cuvidMapVideoFrame 返回 CUDA_ERROR_INVALID_CONTEXT。

解决方案 在 cuvidCtxLock 之后, 需要检查并手动推送 CUDA 上下文:

```

static int CUDAAPI HandlePictureDisplay(void* pUserData, CUVIDPARSERDISPINFO* pDispInfo)
{
    auto* ctx = static_cast<NvdecContext*>(pUserData);
    std::lock_guard<std::mutex> lock(ctx->mtx);

    // 1. 获取 CUVID 上下文锁
    CUresult lockRes = cuvidCtxLock(ctx->cuLock, 0);
    if (lockRes != CUDA_SUCCESS) {
        return 0;
    }

    // 2. 检查当前线程的 CUDA 上下文
    CUcontext currentCtx = nullptr;
    cuCtxGetCurrent(&currentCtx);
    bool needPop = false;

    if (currentCtx != ctx->cuCtx) {
        // 3. 如果上下文不匹配, 手动推送
        CUresult pushRes = cuCtxPushCurrent(ctx->cuCtx);
        if (pushRes != CUDA_SUCCESS) {
            cuvidCtxUnlock(ctx->cuLock, 0);
            return 0;
        }
        needPop = true;
    }

    // 4. 使用 cleanup lambda 确保正确的清理顺序
    auto cleanup = [&]() {
        if (needPop) {
            CUcontext poppedCtx;
            cuCtxPopCurrent(&poppedCtx);
        }
        cuvidCtxUnlock(ctx->cuLock, 0);
    };

    // 5. 现在可以安全调用 cuvidMapVideoFrame
    CUresult res = cuvidMapVideoFrame(ctx->decoder, ...);

    // ... 处理帧 ...

    cleanup();
    return 1;
}

```

关键知识点更新

函数	作用	是否绑定上下文
cuvidCtxLock	获取 CUVID 解码器的互斥锁	☒ 否
cuCtxPushCurrent	将上下文推送到当前线程栈	☒ 是
cuCtxSetCurrent	设置当前线程的上下文	☒ 是

正确的多线程 CUVID 操作流程: 1. cuvidCtxLock()-获取锁, 防止并发访问 2. cuCtxGetCurrent()-检查当前线程上下文 3. cuCtxPushCurrent()-如需要, 推送正确的上下文 4. CUVID 操作(cuvidMapVideoFrame 等) 5. cuCtxPopCurrent()-如果推送了, 弹出上下文 6. cuvidCtxUnlock()-释放锁

修改的文件

- Assets/Plugins/NativeVideoPlugin/NvdecStub.cpp
 - HandlePictureDisplay() 添加详细错误日志
 - HandlePictureDisplay() 在 cuvidCtxLock 后检查并推送 CUDA 上下文
 - nvdec_get_texture() 添加纹理上传计数日志
 - 使用 cleanup lambda 确保正确的资源释放顺序

深入诊断 (2026年1月29日下午更新)

在添加了 cuCtxPushCurrent 逻辑后, 进一步诊断发现了更关键的问题。

诊断日志输出:

```
[NvdecPlugin] HandlePictureDisplay: currentCtx=0x6418622cf5c0, expected=0x6418622cf5c0
[NvdecPlugin] HandlePictureDisplay: currentCtx=0x6418622cf5c0, expected=0x6418622cf5c0
... (上下文匹配, 无需 Push)
[NvdecPlugin] cuvidMapVideoFrame failed: 205
```

关键发现: currentCtx 和 expected 是**完全相同的**! 这意味着: 1. CUDA 上下文已经正确绑定到 CUVID 回调线程 2. cuCtxPushCurrent 没有被执行 (因为上下文已经匹配) 3. **但 cuvidMapVideoFrame 仍然返回错误 205**

真正的问题根源 经过进一步分析, 发现问题出在 **HandleVideoSequence** 和 **HandlePictureDecode** 回调没有使用 **cuvidCtxLock**!

回调调用顺序:

Parser 内部线程:

```
HandleVideoSequence()    ← 没有 cuvidCtxLock!
    → cuvidCreateDecoder() ← 可能在错误的上下文状态下创建
HandlePictureDecode()    ← 没有 cuvidCtxLock!
    → cuvidDecodePicture() ← 可能失败
HandlePictureDisplay()   ← 有 cuvidCtxLock
    → cuvidMapVideoFrame() ← 失败: 解码器状态不正确
```

虽然 HandlePictureDisplay 中使用了 cuvidCtxLock, 但如果解码器是在没有正确锁保护的情况下创建的, 解码器本身可能处于不

完整解决方案 所有 CUVID 回调都必须使用 cuvidCtxLock:

```
// HandleVideoSequence - 解码器创建
static int CUDAAPI HandleVideoSequence(void* pUserData, CUVIDEOFORMAT* pFormat)
{
    auto* ctx = static_cast<NvdecContext*>(pUserData);
    std::lock_guard<std::mutex> lock(ctx->mtx);

    // 获取 CUVID 上下文锁
    CUresult lockRes = cuvidCtxLock(ctx->cuLock, 0);
    if (lockRes != CUDA_SUCCESS) {
        return 0;
    }

    // ... 解码器创建代码 ...

    cuvidCtxUnlock(ctx->cuLock, 0);
    return 1;
}

// HandlePictureDecode - 解码操作
static int CUDAAPI HandlePictureDecode(void* pUserData, CUVIDPICPARAMS* pPicParams)
```

```

{
    auto* ctx = static_cast<NvdecContext*>(pUserData);
    if (!ctx->decoder) return 0;

    CUresult lockRes = cuvidCtxLock(ctx->cuLock, 0);
    if (lockRes != CUDA_SUCCESS) {
        return 0;
    }

    CUresult res = cuvidDecodePicture(ctx->decoder, pPicParams);
    cuvidCtxUnlock(ctx->cuLock, 0);

    if (res != CUDA_SUCCESS) {
        return 0;
    }
    ctx->frames_decoded++;
    return 1;
}

// HandlePictureDisplay - 帧显示 (已有锁)
static int CUDAAPI HandlePictureDisplay(void* pUserData, CUVIDPARSERDISPINFO* pDispInfo)
{
    // ... 已有 cuvidCtxLock 逻辑 ...
}

```

CUVID 回调完整列表

回调函数	调用时机	需要 cuvidCtxLock
HandleVideoSequence	检测到新的视频序列/分辨率变化	☒ 必须
HandlePictureDecode	有新的图片需要解码	☒ 必须
HandlePictureDisplay	有解码完成的图片可以显示	☒ 必须

关键教训 所有 CUVID 解析器回调都在解析器内部线程中执行，而不是创建解析器的线程。因此：

1. 每个回调都需要 cuvidCtxLock 来确保线程安全
2. 不能假设任何回调天然具有正确的 CUDA 上下文
3. cuvidCtxLock 不仅是互斥锁，它还可能影响 CUDA 驱动内部的上下文状态管理

修改的文件（更新）

- Assets/Plugins/NativeVideoPlugin/NvdecStub.cpp
 - HandleVideoSequence() 添加 cuvidCtxLock/Unlock
 - HandlePictureDecode() 添加 cuvidCtxLock/Unlock
 - HandlePictureDisplay() 保持现有的锁逻辑
 - 所有回调使用 fprintf(stderr, ...) 输出日志

9. cuvidMapVideoFrame 32位/64位函数签名不匹配

问题描述

在添加了所有 CUVID 回调的 cuvidCtxLock 后，cuvidMapVideoFrame 仍然持续返回错误 205，但前 2 帧可以成功处理。

症状

- 解码器创建成功: [NVDEC] Decoder created: 1280x720, codec=8
- 前 2 帧成功: decoded=2, displayed=2
- 后续帧全部失败: cuvidMapVideoFrame failed: 205
- 上下文检查显示正确: currentCtx=0x..., expected=0x... (两者相同)

诊断日志

```
[NVDEC] CUDA-GL registration SUCCESS
[NVDEC] Decoder created: 1280x720, codec=8
[NvdecPlugin] HandlePictureDisplay: currentCtx=0x62ff075faf30, expected=0x62ff075faf30
[NvdecPlugin] HandlePictureDisplay: currentCtx=0x62ff075faf30, expected=0x62ff075faf30
[NVDEC] Texture uploaded (count=1, decoded=2, displayed=2)
[NvdecPlugin] cuvidMapVideoFrame failed: 205
[NvdecPlugin] cuvidMapVideoFrame failed: 205
... (持续失败)
```

原因分析

CUVID 函数的 32 位和 64 位版本 NVIDIA Video Codec SDK 提供了两个版本的帧映射函数:

函数名	指针类型	适用场景
cuvidMapVideoFrame	unsigned int *pDevPtr	32 位系统或旧驱动
cuvidMapVideoFrame64	unsigned long long *pDevPtr	64 位系统 (现代系统)

类型定义 (来自 dynlink_cuviddec.h) :

// 32 位版本

```
typedef CUDAAPI tcuvidMapVideoFrame(CUvideodecoder hDecoder, int nPicIdx,
                                     unsigned int *pDevPtr, unsigned int *pPitch,
                                     CUVIDPROC_PARAMS *pVPP);
```

// 64 位版本

```
typedef CUDAAPI tcuvidMapVideoFrame64(CUvideodecoder hDecoder, int nPicIdx,
                                       unsigned long long *pDevPtr,
                                       unsigned int *pPitch, CUVIDPROC_PARAMS *pVPP);
```

// 头文件中的宏定义 (用于源码兼容)

```
#define tcuvidMapVideoFrame tcuvidMapVideoFrame64
```

问题根因 我们的代码中: 1. 函数指针类型 tcuvidMapVideoFrame 被宏定义为 tcuvidMapVideoFrame64 (64 位版本) 2. 我们传递的是 CUdeviceptr (在 64 位系统上是 unsigned long long) 3. **但是**, 我们使用 dlsym(handle, "cuvidMapVideoFrame") 加载符号 4. 这加载的是 **32 位版本** 的函数!

结果: - 函数签名不匹配 - 64 位指针被截断或错误解释 - 导致 CUDA_ERROR_INVALID_CONTEXT (205) 或其他未定义行为

解决方案

修改函数加载逻辑, 优先加载 64 位版本:

```
#define LOAD_FUNC(name) \
    name = (t##name*)dlsym(handle, #name); \
    if (!name) { fprintf(stderr, "[NVDEC] Failed to load " #name "\n"); return false; }
```

// 新增: 优先加载 64 位版本的宏

```
#define LOAD_FUNC_64(name) \
    name = (t##name*)dlsym(handle, #name "64"); \
    if (!name) name = (t##name*)dlsym(handle, #name); \
    if (!name) { fprintf(stderr, "[NVDEC] Failed to load " #name " or " #name "64\n"); return false; }

// 使用示例
LOAD_FUNC(cuvidCreateVideoParser)
LOAD_FUNC(cuvidDecodePicture)
LOAD_FUNC_64(cuvidMapVideoFrame) // 优先加载 cuvidMapVideoFrame64
LOAD_FUNC_64(cuvidUnmapVideoFrame) // 优先加载 cuvidUnmapVideoFrame64
LOAD_FUNC(cuvidDestroyDecoder)
```

加载顺序

LOAD_FUNC_64 宏的加载顺序：1. 先尝试 dlsym(handle, "cuvidMapVideoFrame64") - 64 位版本 2. 如果失败，尝试 dlsym(handle, "cuvidMapVideoFrame") - 32 位版本（兼容旧驱动） 3. 如果都失败，报错并返回 false

关键知识点

- 符号名 vs 类型名：**
 - C 预处理器的 #define 只影响源码中的类型名
 - 动态链接库中的符号名是独立的
 - dlsym 需要使用**实际的符号名**，不受 #define 影响
- CUDA/NVDEC 的 64 位支持：**
 - 现代 NVIDIA 驱动（500+）同时提供 32 位和 64 位版本的 API
 - 64 位系统应优先使用 *64 后缀的函数
 - 这确保设备指针（CUdeviceptr）被正确处理
- 错误 205 的多种成因：**
 - 上下文未绑定到当前线程
 - 函数签名不匹配（如本问题）
 - 解码器状态不一致

修改的文件

- Assets/Plugins/NativeVideoPlugin/NvdecStub.cpp
 - 添加 LOAD_FUNC_64 宏
 - cuvidMapVideoFrame 改用 LOAD_FUNC_64 加载
 - cuvidUnmapVideoFrame 改用 LOAD_FUNC_64 加载

10. 函数指针类型与实际加载函数签名不匹配

问题描述

即使使用 LOAD_FUNC_64 宏成功加载了 cuvidMapVideoFrame64 函数，视频仍然黑屏，cuvidMapVideoFrame 调用持续返回错误 2 (CUDA_ERROR_OUT_OF_MEMORY) 和错误 205 (CUDA_ERROR_INVALID_CONTEXT)。

症状

```
[NVDEC] Video format: 2560x1440, min_decode_surfaces=6, using=10
[NVDEC] Decoder created: 2560x1440, codec=8
[NvdecPlugin] cuvidMapVideoFrame: decoder=0x58e571156a80, picture_index=0, cuLock=0x58e563ef3470
[NvdecPlugin] cuvidMapVideoFrame: decoder=0x58e571156a80, picture_index=1, cuLock=0x58e563ef3470
[NvdecPlugin] cuvidMapVideoFrame failed: 2 # OUT_OF_MEMORY
[NvdecPlugin] cuvidMapVideoFrame failed: 2
```

```
[NvdecPlugin] cuvidMapVideoFrame failed: 700 # ILLEGAL_ADDRESS
[NvdecPlugin] cuvidMapVideoFrame failed: 205 # INVALID_CONTEXT (之后持续)
```

关键现象：- picture_index=0 成功（无错误日志） - picture_index=1 立即失败，错误 2 (OUT_OF_MEMORY) - 随后出现错误 700 (ILLEGAL_ADDRESS) - 最终稳定在错误 205 (INVALID_CONTEXT)

原因分析

问题 9 的修复不完整 问题 9 中，我们使用 LOAD_FUNC_64 宏加载了 64 位版本的函数：

```
#define LOAD_FUNC_64(name) \
    name = (t##name*)dlsym(handle, #name "64"); \
    if (!name) name = (t##name*)dlsym(handle, #name); \
    ...
```

但函数指针的类型声明仍然是 32 位版本：

```
// 问题代码
tcuvidMapVideoFrame *cuvidMapVideoFrame = nullptr; // 32 位类型!
```

类型签名对比

版本	类型名	pDevPtr 参数类型
32 位	tcuvidMapVideoFrame	unsigned int * (4 字节)
64 位	tcuvidMapVideoFrame64	unsigned long long * (8 字节)

调用时的问题

```
CUdeviceptr dpSrcFrame = 0; // 64 位类型 (8 字节)
cuvidMapVideoFrame(decoder, idx, &dpSrcFrame, &pitch, &vpp);
```

由于函数指针类型是 tcuvidMapVideoFrame (32 位签名)，编译器认为 pDevPtr 参数是 unsigned int*。但实际加载的是 cuvidMapVideoFrame64 (期望 unsigned long long*)。

结果： - 函数内部向 pDevPtr 写入 8 字节的 64 位地址 - 但调用约定认为只有 4 字节 - 导致栈损坏或地址截断 - 返回的 GPU 内存地址无效，后续操作失败

解决方案

修改函数指针类型声明为 64 位版本：

```
// 修复后
// 使用 64 位版本的函数指针类型 (CUdeviceptr 是 64 位)
tcuvidMapVideoFrame64 *cuvidMapVideoFrame = nullptr;
tcuvidUnmapVideoFrame64 *cuvidUnmapVideoFrame = nullptr;
```

同时修改加载宏：

```
// 专门加载 64 位版本的 Map/Unmap 函数 (使用 64 位类型)
#define LOAD_FUNC_MAP64(name) \
    name = (t##name##64*)dlsym(handle, #name "64"); \
    if (!name) { \
        fprintf(stderr, "[NVDEC] Failed to load " #name "64, falling back to 32-bit\n"); \
        name = (t##name##64*)dlsym(handle, #name); \
    } \
    if (!name) { fprintf(stderr, "[NVDEC] Failed to load " #name "\n"); return false; }
```

```
// 使用
LOAD_FUNC_MAP64(cuvidMapVideoFrame)
LOAD_FUNC_MAP64(cuvidUnmapVideoFrame)
```

完整修改

```
// NvdecStub.cpp

// 函数指针声明 - 使用 64 位类型
tcuvidCreateVideoParser *cuvidCreateVideoParser = nullptr;
tcuvidParseVideoData *cuvidParseVideoData = nullptr;
tcuvidDestroyVideoParser *cuvidDestroyVideoParser = nullptr;
tcuvidCreateDecoder *cuvidCreateDecoder = nullptr;
tcuvidDecodePicture *cuvidDecodePicture = nullptr;
tcuvidMapVideoFrame64 *cuvidMapVideoFrame = nullptr; // 64 位类型!
tcuvidUnmapVideoFrame64 *cuvidUnmapVideoFrame = nullptr; // 64 位类型!
tcuvidDestroyDecoder *cuvidDestroyDecoder = nullptr;
// ... 其他函数
```

关键知识点

1. **C 类型转换不改变调用约定:**
 - (t##name*)dlsym(...) 只是告诉编译器如何解释指针
 - 实际调用时的参数传递方式由**函数指针的声明类型**决定
 - 加载的函数和声明的类型必须匹配
2. **ABI 兼容性:**
 - 32 位和 64 位函数的参数传递方式可能不同
 - x86-64 调用约定中, 前几个整数参数通过寄存器传递
 - 指针大小的差异会导致严重问题
3. **错误码含义:**
 - 错误 2 (CUDA_ERROR_OUT_OF_MEMORY): 通常表示地址无效
 - 错误 700 (CUDA_ERROR_ILLEGAL_ADDRESS): 访问了非法 GPU 内存地址
 - 错误 205 (CUDA_ERROR_INVALID_CONTEXT): 上下文被破坏

修改的文件

- Assets/Plugins/NativeVideoPlugin/NvdecStub.cpp
 - 函数指针 cuvidMapVideoFrame 类型从 tcuvidMapVideoFrame* 改为 tcuvidMapVideoFrame64*
 - 函数指针 cuvidUnmapVideoFrame 类型从 tcuvidUnmapVideoFrame* 改为 tcuvidUnmapVideoFrame64*
 - 新增 LOAD_FUNC_MAP64 宏专门用于加载 64 位映射函数

11. GL对象尺寸与视频实际尺寸不匹配

问题描述

即使修复了函数指针类型问题 (问题10), 视频仍然黑屏。诊断日志显示 PBO 尺寸与视频实际尺寸不匹配, 导致 CUDA 核函数写入越界内存。

症状

```
[NVDEC] Video format: 2560x1440, min_decode_surfaces=6, using=10
[NVDEC] Decoder created: 2560x1440, codec=8
[NVDEC] NV12->RGB: src_pitch=2560, pbo_size=6220800, wxh=2560x1440, expected=11059200
[NVDEC] cudaStreamSynchronize failed: 700 (an illegal memory access was encountered)
[NvdecPlugin] cuvidMapVideoFrame failed: 2 # 后续帧 OUT_OF_MEMORY
[NvdecPlugin] cuvidMapVideoFrame failed: 700 # ILLEGAL_ADDRESS
[NvdecPlugin] cuvidMapVideoFrame failed: 205 # INVALID_CONTEXT (持续)
```

关键数据: - **视频尺寸**: 2560 × 1440 - **期望 PBO 大小**: 2560 × 1440 × 3 = 11,059,200 字节 - **实际 PBO 大小**: 1920 × 1080 × 3 = 6,220,800 字节

原因分析

GL 对象创建时机问题

```
// nvdec_get_texture 中的代码
if (!mut_ctx.gl_ready || mut_ctx.tex == 0 || mut_ctx.pbo == 0) {
    int w = mut_ctx.width > 0 ? mut_ctx.width : 1920; // 问题!
    int h = mut_ctx.height > 0 ? mut_ctx.height : 1080;
    setup_gl_objects(mut_ctx, w, h);
}
```

时序问题： 1. Unity 调用 nvdec_get_texture() 获取纹理 2. 此时 ctx.width/height 还是 0（视频数据未到达）
3. 使用默认值 1920×1080 创建 PBO 和纹理 4. 稍后 HandleVideoSequence 回调更新 ctx.width/height 为 2560×1440 5. 但 GL 对象已创建，不会重新创建 6. CUDA 核函数按 2560×1440 写入数据到 1920×1080 的 PBO
7. **内存越界**，CUDA 上下文损坏

内存计算

尺寸	RGB 大小	差异
1920×1080	6,220,800 字节	基准
2560×1440	11,059,200 字节	+78%

CUDA 核函数尝试写入 11MB 数据到 6MB 的缓冲区，导致越界访问。

解决方案

1. 添加 GL 对象尺寸追踪

```
// NvdecStub.h - NvdecContext 结构体
struct NvdecContext {
    // ... 其他字段 ...
    int width = 0; // 视频实际尺寸 (来自 NVDEC)
    int height = 0;

    // GL 对象尺寸追踪
    int gl_width = 0; // GL 对象创建时的尺寸
    int gl_height = 0;
    // ...
};
```

2. 添加 GL 对象销毁函数

```
static void destroy_gl_objects(NvdecContext& ctx)
{
    if (ctx.cuPbo) {
        cuGraphicsUnregisterResource(ctx.cuPbo);
        ctx.cuPbo = nullptr;
    }
    if (ctx.pbo) {
        glDeleteBuffers(1, &ctx.pbo);
        ctx.pbo = 0;
    }
    if (ctx.tex) {
        glDeleteTextures(1, &ctx.tex);
        ctx.tex = 0;
    }
}
```

```

    ctx.gl_ready = false;
}

```

3. 修改 setup_gl_objects 支持尺寸变化

```

static int setup_gl_objects(NvdecContext& ctx, int w, int h)
{
    // 如果尺寸变化，需要重新创建
    if (ctx.gl_ready && (ctx.gl_width != w || ctx.gl_height != h)) {
        fprintf(stderr, "[NVDEC] Size changed from %dx%d to %dx%d, recreating GL objects\n",
            ctx.gl_width, ctx.gl_height, w, h);
        destroy_gl_objects(ctx);
    }

    if (ctx.gl_ready) return 0;

    ctx.gl_width = w;
    ctx.gl_height = h;

    // 创建 PBO 和纹理...
}

```

4. 修改 nvdec_get_texture 等待视频尺寸

```

int nvdec_get_texture(const NvdecContext& ctx)
{
    // 获取当前视频尺寸
    int w = mut_ctx.width;
    int h = mut_ctx.height;

    // 如果视频尺寸还未知，等待
    if (w <= 0 || h <= 0) {
        return 0; // 返回 0，不创建 GL 对象
    }

    // 创建或重新创建 GL 对象（如果尺寸变化）
    if (!mut_ctx.gl_ready || mut_ctx.gl_width != w || mut_ctx.gl_height != h) {
        setup_gl_objects(mut_ctx, w, h);
    }
    // ...
}

```

关键知识点

1. 延迟初始化:

- GL 对象应在视频尺寸确定后再创建
- 不应使用默认尺寸，而应等待 NVDEC 回调

2. 动态尺寸支持:

- 视频流的尺寸可能在运行时变化
- GL 对象需要支持重新创建

3. CUDA 内存安全:

- CUDA 核函数的输出缓冲区大小必须足够
- 越界写入会导致 CUDA_ERROR_ILLEGAL_ADDRESS (700)
- 一旦发生越界，CUDA 上下文会被破坏，后续所有操作都会失败

4. 错误传播:

- 错误 700 后变成错误 205

- 这是因为非法内存访问破坏了 CUDA 上下文
- 后续操作都会返回 “无效上下文” 错误

修改的文件

- Assets/Plugins/NativeVideoPlugin/NvdecStub.h
 - 添加 gl_width 和 gl_height 字段
- Assets/Plugins/NativeVideoPlugin/NvdecStub.cpp
 - 添加 destroy_gl_objects() 函数
 - 修改 setup_gl_objects() 支持尺寸变化检测和重新创建
 - 修改 nvdec_get_texture() 等待视频尺寸已知后再创建 GL 对象
 - 修改 glTexSubImage2D 调用使用 gl_width/gl_height

文档最后更新：2026年1月29日